Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

// Divide and Conquer: find the first occurrence of 0
int firstZero(int arr[], int low, int high)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == 0)
    {
        // Check whether this is the first zero
        if (mid == 0 || arr[mid - 1] == 1)
            return mid;

        return firstZero(arr, low, mid - 1);
    }
    else
    {
        return firstZero(arr, mid + 1, high);
    }
}

int main()
{
    int m;
    scanf("%d", &m);

    int arr[m];

    for (int i = 0; i < m; i++)
        scanf("%d", &arr[i]);

    int index = firstZero(arr, 0, m - 1);

    if (index == -1)
        printf("0");
    else
        printf("%d", m - index);

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int count(int arr[], int left, int right, int num)
{
    int c = 0;

    for (int i = left; i <= right; i++)
    {
        if (arr[i] == num)
            c++;
    }

    return c;
}

int majorityElement(int arr[], int left, int right)
{
    // Only one element
    if (left == right)
        return arr[left];

    int mid = (left + right) / 2;

    // Find majority element in left and right halves
    int leftMajority = majorityElement(arr, left, mid);
    int rightMajority = majorityElement(arr, mid + 1, right);

    // If both halves have the same majority element
    if (leftMajority == rightMajority)
        return leftMajority;

    // Count both candidates in the complete range
    int leftCount = count(arr, left, right, leftMajority);
    int rightCount = count(arr, left, right, rightMajority);

    if (leftCount > rightCount)
        return leftMajority;
    else
        return rightMajority;
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("%d", majorityElement(arr, 0, n - 1));
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == x)
        return arr[mid];

    if (arr[mid] < x)
    {
        int result = findFloor(arr, mid + 1, high, x);

        if (result == -1)
            return arr[mid];

        return result;
    }
    else
    {
        return findFloor(arr, low, mid - 1, x);
    }
}

int main()
{
    int n, x;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &x);

    int result = findFloor(arr, 0, n - 1, x);

    printf("%d", result);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int arr[], int left, int right, int x)
{
    // No pair exists
    if (left >= right)
        return 0;

    int sum = arr[left] + arr[right];

    // Pair found
    if (sum == x)
    {
        printf("%d\n", arr[left]);
        printf("%d\n", arr[right]);
        return 1;
    }

    // Sum is smaller, move left pointer
    if (sum < x)
        return findPair(arr, left + 1, right, x);

    // Sum is larger, move right pointer
    return findPair(arr, left, right - 1, x);
}

int main()
{
    int n, x;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &x);

    if (!findPair(arr, 0, n - 1, x))
        printf("No");

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Co

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int i = low;
        int j = high;
        int pivot = arr[low];

        while (i < j)
        {
            while (arr[i] <= pivot && i < high)
                i++;

            while (arr[j] > pivot)
                j--;

            if (i < j)
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[low];
        arr[low] = arr[j];
        arr[j] = temp;

        quickSort(arr, low, j - 1);
        quickSort(arr, j + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.